Dr. Mundo - Deployment Guide

Hosting on a Proxmox-managed Linux Container (DLSU CCS Cloud)

This guide walks the deployer through hosting Dr. Mundo on the provisioned Proxmox LXC. It builds on the reference document *"Configuring Local SSH Access to Provisioned Linux Containers"* (K. Kalaw, DLSU CCS) for the Proxmox login, user creation, and SSH steps. Follow the sections in order.

Prerequisites

- The provisioned Proxmox and LXC credentials (see the table below; passwords were sent to you separately).
- An **OpenAI API key** - the app calls gpt-4o-mini and the embedding model at runtime.
- An SSH client on your local machine (check with `ssh -V`).

Provisioned environment

Proxmox	Value
Web console	https://ccscloud.dlsu.edu.ph/
Username	STAI10014
Realm	Proxmox VE
Node	Mayari

Linux Container (LXC)	Value
Container ID	11549
Name	STAI10014-Server
Operating system	Ubuntu 22.04 LTS
Root username	root
Private IP	10.2.0.45/16
Public IP	103.231.240.130

Port forwarding (read this first)

The container's **internal** ports are NAT-forwarded to **external** ports on the public IP. Only ports 22, 7860, and 8000 are forwarded, so the services must bind to those internal ports. In particular, run the Streamlit UI on **7860** (not its default 8501).

Purpose	Internal	External	Access from your machine / browser
SSH	22	2133	`ssh -p 2133 <user>@103.231.240.130`
FastAPI (API)	8000	2173	`http://103.231.240.130:2173/docs`
Streamlit (UI)	7860	2153	`http://103.231.240.130:2153`

Step 1 - Start the container in Proxmox

Log in to https://ccscloud.dlsu.edu.ph/ with the Proxmox **Username**, **Password**, and **Realm = Proxmox VE**. Dismiss the “No valid subscription” dialog. In the left sidebar open node **Mayari** and select container **11549 (STAI10014-Server)**. New containers are **stopped** by default - click **Start**.

Step 2 - Create a Linux user with sudo (one time)

Root SSH is not permitted, so create a normal user through the Proxmox **Console** (log in there with the LXC root credentials first). This follows the reference SSH guide.

```
# in the Proxmox web Console, logged in as root:
adduser your_username           # set a password; other fields can be left blank
usermod -aG sudo your_username   # grant sudo
exit
```

If a user was already created for your group, you can skip this step.

Step 3 - SSH into the container

From your **local** terminal (accept the host key with `yes` on first connect, then enter the password):

```
ssh -p 2133 your_username@103.231.240.130
```

Step 4 - Install system dependencies

Ubuntu 22.04 ships Python 3.10; install Python 3.11 to match the tested environment. Run everything below **inside the container** (over SSH).

```
sudo apt update
sudo apt install -y software-properties-common git tmux
sudo add-apt-repository -y ppa:deadsnakes/ppa
sudo apt update
sudo apt install -y python3.11 python3.11-venv
```

Step 5 - Clone the repository

```
git clone https://github.com/Dawsxn/DrMundo.git
cd DrMundo
```

Step 6 - Create a virtual environment and install

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Step 7 - Build the local database

Builds dr_mundo.db from the committed CSVs. The catalog embeddings (data/embeddings.npz) are already committed, so no API key is needed here.

```
python data/load_db.py
```

Step 8 - Configure the API key

```
cp .env.example .env
nano .env        # set OPENAI_API_KEY=sk-...  then save (Ctrl-O, Enter, Ctrl-X)
```

Step 9 - Run the two services

Use **tmux** so the services keep running after you disconnect. Start a session, run the API, open a second window (Ctrl-b then c) for the UI on port **7860**, then detach with Ctrl-b then d.

```
tmux new -s drmundo
# --- window 1: API ---
source .venv/bin/activate
uvicorn api.main:app --host 0.0.0.0 --port 8000

# Ctrl-b c  to open window 2: UI
source .venv/bin/activate
streamlit run ui/app.py --server.port 7860 --server.address 0.0.0.0 --server.headless true

# Ctrl-b d  to detach (services keep running).  Reattach: tmux attach -t drmundo
```

The UI reaches the API at http://localhost:8000 by default (both run in the same container), so no extra configuration is needed.

Step 10 - Open the app

- **Web UI:** http://103.231.240.130:2153
- **API docs (Swagger):** http://103.231.240.130:2173/docs
- **Health check:** http://103.231.240.130:2173/health

Quick sanity check from inside the container:

```
curl http://localhost:8000/health      # -> {"status":"ok","database":true,"embeddings":true}
```

Appendix A - Run as systemd services (recommended for persistence)

tmux is fine for a demo, but systemd restarts the app automatically after a crash or reboot. The app loads .env itself, so the units only need the working directory. Replace USER with your username in all three places.

Create /etc/systemd/system/drmundo-api.service:

```
[Unit]
Description=Dr. Mundo API (FastAPI)
After=network.target

[Service]
User=USER
WorkingDirectory=/home/USER/DrMundo
ExecStart=/home/USER/DrMundo/.venv/bin/uvicorn api.main:app --host 0.0.0.0 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target
```

Create /etc/systemd/system/drmundo-ui.service:

```
[Unit]
Description=Dr. Mundo UI (Streamlit)
After=drmundo-api.service

[Service]
User=USER
WorkingDirectory=/home/USER/DrMundo
ExecStart=/home/USER/DrMundo/.venv/bin/streamlit run ui/app.py \
  --server.port 7860 --server.address 0.0.0.0 --server.headless true
Restart=always

[Install]
WantedBy=multi-user.target
```

Enable and start both:

```
sudo systemctl daemon-reload
sudo systemctl enable --now drmundo-api drmundo-ui
sudo systemctl status drmundo-api drmundo-ui     # check they are active
journalctl -u drmundo-api -f                     # follow API logs
```

Appendix B - Docker alternative

The repository includes a Dockerfile and docker-compose.yml. On this LXC, the native deployment above is recommended, because Docker inside an LXC requires container nesting to be enabled on the host. If Docker is available, remember to publish the UI on **7860** (the compose file uses 8501 for local use, so change the ui service's port and command to 7860 before running `docker compose up --build`).

Appendix C - Troubleshooting

Symptom	Fix
SSH refused / times out	Make sure the container is Started in Proxmox; connect on port 2133 as your new user (not root).
App not reachable from a browser	Bind to 0.0.0.0 (not 127.0.0.1) and use internal ports 7860/8000; browse the external ports 2153/2173.

Symptom	Fix
“OPENAI_API_KEY is not set” / 500 on /ask	Ensure .env contains a valid OPENAI_API_KEY, then restart the service(s).
UI says it cannot reach the API	Start the API (uvicorn) first; it must be listening on port 8000 in the same container.
python3.11: command not found	Install it via the deadsnakes PPA (Step 4).
Services stop when you log out	Use tmux (Step 9) or systemd services (Appendix A) so they persist.

Dr. Mundo · See the companion “Project Summary” PDF for what the app does. Estimates only - not medical or financial advice.